Question **19**

Answer saved

Marked out of 9.00

Remove flag

Consider the class `Circle`:

```
const double PI = 3.1415927;

class Circle

{

public:

    Circle (string name, double rad);

    double area () const;

    string getShapeName() const;

    double getRadius()const;

private:

     double radius;

     string shapeName;

};
```

(a) Derive a class `Cylinder` from the class `Circle`. Provide only the interface for the class `Cylinder`. The class `Cylinder` has an additional private double member variable `height`, as well as an additional member function `volume()`. Include an accessor for the member variable `height`. Redefine the member function `area()` for the class `Cylinder`. (6)

(b) Implement only the overloaded constructor for the class `Cylinder`. (3)

(a) Derive a class `Cylinder`

(b) Implement the overloaded constructor for the class `Cylinder`

Question **20**

Answer saved

Marked out of 1.00

Flag question

Given the declarations

```
class Pet //implemented as an ADT

{

public:

   string getColour();

   string getType();

//other member functions

private:

   string name;

   string colour;

   int age;

};

 int nrPets = 30;

Pen myPets[nrPets];
```

To display the names of all the **brown** pets in the array `myPets` on the screen, we use the statement

```
    for (int i = 0; i <= nrPets; i++ )

        if (myPets.getColour(i)== "brown" )

            cout << myPets.getName(i) << endl;
```

Select one:

- True
- False

Previous page

Next page